

# **LAPORAN TUGAS 1**

## **SISTEM OPERASI**



**Dosen Pengampu Mata Kuliah:**

**Triawan Adi Cahyanto, M.Kom**

**Disusun Oleh:**

**Suci Nur Hidayati (NIM 2410651023)**

**PRODI TEKNIK INFORMATIKA**

**FAKULTAS TEKNIK**

**UNIVERSITAS MUHAMMADIYAH JEMBER**

**2025**

## Praktik 1.1 – Menjelajahi Layanan OS

Pada praktik ini dipelajari berbagai layanan dasar yang disediakan oleh sistem operasi, seperti layanan eksekusi program, manajemen file, I/O, serta manajemen proses. Tujuannya agar memahami peran utama sistem operasi dalam mengelola sumber daya komputer dan menyediakan antarmuka antara perangkat keras dan pengguna. Mahasiswa menulis program C sederhana untuk memanggil layanan-layanan tersebut dan mengamati hasilnya melalui terminal.

```
suci@LAPTOP-2K58MOR4:~/praktikum_os$ ~/praktikum_os
-bash: /home/suci/praktikum_os: Is a directory
suci@LAPTOP-2K58MOR4:~/praktikum_os$ ls
custom_shell  file_monitor  file_monitor.log  os_services  process_manager  system_info  test_file.txt
custom_shell.c  file_monitor.c  monitor_dir      os_services.c  process_manager.c  system_info.c
suci@LAPTOP-2K58MOR4:~/praktikum_os$ nano simple_shell.c
suci@LAPTOP-2K58MOR4:~/praktikum_os$ gcc -o simple_shell simple_shell.c
suci@LAPTOP-2K58MOR4:~/praktikum_os$ ls
custom_shell  file_monitor.c  os_services  process_manager.c  system_info
custom_shell.c  file_monitor.log  os_services.c  simple_shell      system_info.c
file_monitor  monitor_dir     process_manager  simple_shell.c    test_file.txt
suci@LAPTOP-2K58MOR4:~/praktikum_os$ ./simple_shell
mysHELL> ls
custom_shell  file_monitor.c  os_services  process_manager.c  system_info
custom_shell.c  file_monitor.log  os_services.c  simple_shell      system_info.c
file_monitor  monitor_dir     process_manager  simple_shell.c    test_file.txt
mysHELL> pwd
/home/suci/praktikum_os
mysHELL> echo Halo Dunia
Halo Dunia
mysHELL> exit
suci@LAPTOP-2K58MOR4:~/praktikum_os$ |
```

---

## Praktik 1.2 – Program Execution Service

Percobaan ini menunjukkan bagaimana sistem operasi mengeksekusi suatu program dan memberikan identitas unik pada setiap proses yang berjalan. Dengan menggunakan fungsi `getpid()` dan `getppid()`, mahasiswa dapat mengetahui Process ID (PID) dan Parent Process ID (PPID). Praktik ini penting untuk memahami hubungan hierarki antar proses di dalam sistem operasi.

---

## Praktik 2.1 – File System Service

Mahasiswa mempelajari cara sistem operasi mengatur dan mengelola file menggunakan fungsi standar C seperti `fopen()`, `fprintf()`, `fscanf()`, dan `remove()`. Melalui program sederhana, pengguna dapat membuat, menulis, membaca, serta menghapus file. Hal ini membantu memahami konsep *file descriptor* dan bagaimana kernel menangani operasi file di tingkat sistem

---

## Praktik 3.1 – I/O Service

Praktik ini menjelaskan bagaimana sistem operasi menangani operasi Input/Output, baik dari keyboard, layar, maupun perangkat lainnya. Program menggunakan fungsi seperti `scanf()` dan `printf()` untuk membaca input pengguna dan menampilkan output ke layar. Mahasiswa belajar bagaimana OS menjadi perantara antara program dan perangkat keras.

```
suci@LAPTOP-2K58M0R4:~/praktikum_os$ ./process_syscalls
=== Process Control System Calls ===
Parent PID: 2085
Parent waiting for child (PID: 2086)...
Child PID: 2086
Child's Parent PID: 2085
Child executing 'ls' command...
total 196
-rwxr-xr-x 1 suci suci 17136 Oct 10 13:17 custom_shell
-rw-r--r-- 1 suci suci 4044 Oct 10 13:17 custom_shell.c
-rwxr-xr-x 1 suci suci 17224 Oct 9 22:34 file_monitor
-rw-r--r-- 1 suci suci 6843 Oct 9 22:34 file_monitor.c
-rw-r--r-- 1 suci suci 322 Oct 10 13:02 file_monitor.log
drwxr-xr-x 3 suci suci 4096 Oct 10 13:02 monitor_dir
-rwxr-xr-x 1 suci suci 16488 Oct 10 17:36 os_services
-rw-r--r-- 1 suci suci 1456 Oct 10 17:35 os_services.c
-rwxr-xr-x 1 suci suci 21288 Oct 9 22:27 process_manager
-rw-r--r-- 1 suci suci 10661 Oct 9 22:06 process_manager.c
-rwxr-xr-x 1 suci suci 16360 Oct 12 15:48 process_syscalls
-rw-r--r-- 1 suci suci 831 Oct 12 15:48 process_syscalls.c
-rwxr-xr-x 1 suci suci 16528 Oct 12 13:25 simple_shell
-rw-r--r-- 1 suci suci 1145 Oct 12 13:24 simple_shell.c
-rwxr-xr-x 1 suci suci 16960 Oct 10 13:23 system_info
-rw-r--r-- 1 suci suci 4549 Oct 10 13:23 system_info.c
-rw-r--r-- 1 suci suci 23 Oct 10 17:36 test_file.txt
Child exited with status: 0
```

## Praktik 4.1 – Process Management

Mahasiswa mempelajari bagaimana proses dibuat dan dijalankan secara bersamaan menggunakan fungsi `fork()`. Praktik ini memperlihatkan bagaimana satu program dapat bercabang menjadi dua proses: parent dan child. Hasil output

menunjukkan identitas dan urutan eksekusi keduanya.

```
suci@LAPTOP-2K58MOR4:~/praktikum_os$ nano my_daemon.c
suci@LAPTOP-2K58MOR4:~/praktikum_os$ gcc -o my_daemon my_daemon.c
suci@LAPTOP-2K58MOR4:~/praktikum_os$ ls -l
total 260
-rwxr-xr-x 1 suci suci 17136 Oct 10 13:17 custom_shell
-rw-r--r-- 1 suci suci 4044 Oct 10 13:17 custom_shell.c
-rwxr-xr-x 1 suci suci 17224 Oct 9 22:34 file_monitor
-rw-r--r-- 1 suci suci 6843 Oct 9 22:34 file_monitor.c
-rw-r--r-- 1 suci suci 322 Oct 10 13:02 file_monitor.log
-rwxr-xr-x 1 suci suci 16352 Oct 12 15:51 file_syscalls
-rw-r--r-- 1 suci suci 966 Oct 12 15:51 file_syscalls.c
-rwxr-xr-x 1 suci suci 16384 Oct 12 15:55 info_syscalls
-rw-r--r-- 1 suci suci 711 Oct 12 15:55 info_syscalls.c
drwxr-xr-x 3 suci suci 4096 Oct 10 13:02 monitor_dir
-rwxr-xr-x 1 suci suci 16344 Oct 12 16:03 my_daemon
-rw-r--r-- 1 suci suci 824 Oct 12 16:03 my_daemon.c
-rwxr-xr-x 1 suci suci 16488 Oct 10 17:36 os_services
-rw-r--r-- 1 suci suci 1456 Oct 10 17:35 os_services.c
-rwxr-xr-x 1 suci suci 21288 Oct 9 22:27 process_manager
-rw-r--r-- 1 suci suci 10661 Oct 9 22:06 process_manager.c
-rwxr-xr-x 1 suci suci 16360 Oct 12 15:48 process_syscalls
-rw-r--r-- 1 suci suci 831 Oct 12 15:48 process_syscalls.c
-rwxr-xr-x 1 suci suci 16528 Oct 12 13:25 simple_shell
-rw-r--r-- 1 suci suci 1145 Oct 12 13:24 simple_shell.c
-rw-r--r-- 1 suci suci 25 Oct 12 15:51 syscall_test.txt
-rwxr-xr-x 1 suci suci 16960 Oct 10 13:23 system_info
-rw-r--r-- 1 suci suci 4549 Oct 10 13:23 system_info.c
-rw-r--r-- 1 suci suci 23 Oct 10 17:36 test_file.txt
suci@LAPTOP-2K58MOR4:~/praktikum_os$ sudo mv my_daemon /usr/local/bin/
```

```
suci@LAPTOP-2K58MOR4:~/praktikum_os$ sudo systemctl status my_daemon
● my_daemon.service - My Custom Daemon
   Loaded: loaded (/etc/systemd/system/my_daemon.service; disabled; preset: enabled)
   Active: active (running) since Sun 2025-10-12 16:04:34 WIB; 7s ago
     Process: 2172 ExecStart=/usr/local/bin/my_daemon (code=exited, status=0/SUCCESS)
    Main PID: 2173 (my_daemon)
      Tasks: 1 (limit: 3440)
     Memory: 200.0K (peak: 548.0K)
        CPU: 5ms
    CGroup: /system.slice/my_daemon.service
            └─2173 /usr/local/bin/my_daemon
```

## Praktik 5.1 – Proses Linking

Tujuan dari praktik ini adalah memahami proses *linking*, yaitu tahap penggabungan beberapa file program (.h dan .c) menjadi satu program utuh. Mahasiswa menulis fungsi di file terpisah seperti `math_ops.h` dan `math_ops.c`, lalu mengompilasinya bersama file utama menggunakan *linker*. Konsep ini penting dalam pengembangan perangkat lunak modular.

```

suci@LAPTOP-2K58MOR4:~/praktikum_os$ sudo systemctl stop my_daemon
suci@LAPTOP-2K58MOR4:~/praktikum_os$ nano math_ops.h
suci@LAPTOP-2K58MOR4:~/praktikum_os$ nano math_ops.c
suci@LAPTOP-2K58MOR4:~/praktikum_os$ nano main.c
suci@LAPTOP-2K58MOR4:~/praktikum_os$ gcc -E main.c -o main.i
suci@LAPTOP-2K58MOR4:~/praktikum_os$ gcc -S main.c -o main.s
suci@LAPTOP-2K58MOR4:~/praktikum_os$ gcc -c main.c -o main.o
suci@LAPTOP-2K58MOR4:~/praktikum_os$ gcc main.o math_ops.o -o program
/usr/bin/ld: cannot find math_ops.o: No such file or directory
collect2: error: ld returned 1 exit status
suci@LAPTOP-2K58MOR4:~/praktikum_os$ gcc -c main.c -o main.o
gcc -c math_ops.c -o math_ops.o
suci@LAPTOP-2K58MOR4:~/praktikum_os$ ls
custom_shell      file_syscalls     main.i            math_ops.o        process_manager    simple_shell.c
custom_shell.c    file_syscalls.c   main.o            monitor_dir       process_manager.c  syscall_test.txt
file_monitor      info_syscalls     main.s            my_daemon.c       process_syscalls   system_info
file_monitor.c    info_syscalls.c   math_ops.c        os_services       process_syscalls.c system_info.c
file_monitor.log   main.c            math_ops.h        os_services.c     simple_shell       test_file.txt
suci@LAPTOP-2K58MOR4:~/praktikum_os$ gcc main.o math_ops.o -o program
suci@LAPTOP-2K58MOR4:~/praktikum_os$ ./program
5 + 3 = 8
5 * 3 = 15

```

## Praktik 6.1 – System Calls

Praktik ini menjelaskan perbedaan antara pemanggilan fungsi biasa dan *system call*. Contohnya, fungsi `write()` dan `read()` merupakan *system call* yang berkomunikasi langsung dengan kernel. Dengan `strace`, mahasiswa dapat melihat daftar *system call* yang dijalankan saat program bekerja.

```

suci@LAPTOP-2K58MOR4:~/praktikum_os$ nano os_specific.c
suci@LAPTOP-2K58MOR4:~/praktikum_os$ gcc os_specific.c -o os_specific
./os_specific
=== OS-Specific Application ===
OS: Linux
System: Linux
Release: 6.6.87.2-microsoft-standard-WSL2
Version: #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025

Linux-specific: Process ID = 2226
suci@LAPTOP-2K58MOR4:~/praktikum_os$ file os_specific
readelf -h os_specific
readelf -S os_specific
objdump -d os_specific | head -50
os_specific: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=334d4d381bcb2c27c8d33b5f83d6d9031f808181, for GNU/Linux 3.2.0, not stripped
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:             ELF64
  Data:              2's complement, little endian
  Version:           1 (current)
  OS/ABI:            UNIX - System V
  ABI Version:       0
  Type:              DYN (Position-Independent Executable file)
  Machine:           Advanced Micro Devices X86-64
  Version:           0x1
  Entry point address: 0x10e0
  Start of program headers: 64 (bytes into file)
  Start of section headers: 14200 (bytes into file)
  Flags:             0x0

```



```

Size of this header:      64 (bytes)
Size of program headers: 56 (bytes)
Number of program headers: 13
Size of section headers: 64 (bytes)
Number of section headers: 31
Section header string table index: 30
There are 31 section headers, starting at offset 0x3778:

Section Headers:
[Nr] Name              Type          Address          Offset
     Size              EntSize          Flags Link Info  Align
[ 0]                  NULL              0000000000000000 00000000
     0000000000000000 0000000000000000 0 0 0
[ 1] .interp            PROGBITS         000000000000318 00000318
     000000000000001c 0000000000000000 A 0 0 1
[ 2] .note.gnu.pr[...] NOTE              000000000000338 00000338
     0000000000000030 0000000000000000 A 0 0 8
[ 3] .note.gnu.bu[...] NOTE              000000000000368 00000368
     0000000000000024 0000000000000000 A 0 0 4
[ 4] .note.ABI-tag       NOTE              00000000000038c 0000038c
     0000000000000020 0000000000000000 A 0 0 4
[ 5] .gnu.hash           GNU_HASH          0000000000003b0 000003b0
     0000000000000024 0000000000000000 A 6 0 8
[ 6] .dynsym             DYNSYM            0000000000003d8 000003d8
     0000000000000108 0000000000000018 A 7 1 8
[ 7] .dynstr             STRTAB            0000000000004e0 000004e0
     00000000000000bc 0000000000000000 A 0 0 1
[ 8] .gnu.version         VERSYM            00000000000059c 0000059c
     0000000000000016 0000000000000002 A 6 0 2
[ 9] .gnu.version_r       VERNEED           0000000000005b8 000005b8

```

## Praktik 7.1 – Process Control

Mahasiswa belajar bagaimana sistem operasi mengontrol proses: menghentikan, melanjutkan, atau menunda jalannya proses menggunakan `sleep()`, `kill`, dan `ps`. Praktik ini penting untuk memahami bagaimana kernel mengatur multitasking di sistem operasi modern.

```

0000000000000068 0000000000000008 WA 0 0 8
[25] .data              PROGBITS         0000000000004000 00003000
     0000000000000010 0000000000000000 WA 0 0 8
[26] .bss               NOBITS           0000000000004010 00003010
     0000000000000008 0000000000000000 WA 0 0 1
[27] .comment           PROGBITS         0000000000000000 00003010
     000000000000002b 0000000000000001 MS 0 0 1
[28] .symtab            SYMTAB            0000000000000000 00003040
     000000000000003d8 0000000000000018 29 18 8
[29] .strtab            STRTAB            0000000000000000 00003418
     0000000000000242 0000000000000000 0 0 1
[30] .shstrtab          STRTAB            0000000000000000 0000365a
     000000000000011a 0000000000000000 0 0 1

Key to Flags:
W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
L (link order), O (extra OS processing required), G (group), T (TLS),
C (compressed), x (unknown), o (OS specific), E (exclude),
D (mbind), l (large), p (processor specific)

os_specific:      file format elf64-x86-64

Disassembly of section .init:

0000000000001000 <.init>:
1000:  f3 0f 1e fa          endbr64
1004:  48 83 ec 08          sub    $0x8,%rsp
1008:  48 8b 05 d9 2f 00 00 mov    0x2fd9(%rip),%rax    # 3fe8 <__gmon_start__@Base>
100f:  48 85 c0             test   %rax,%rax
1012:  74 02              je     1016 <.init+0x16>

```

## Praktik 8.1 – Inter Process Communication (IPC)

Mahasiswa mempelajari mekanisme komunikasi antar proses yang berjalan secara bersamaan. Dengan *pipe* atau *shared memory*, dua proses dapat bertukar data tanpa saling mengganggu. Konsep ini menjadi dasar bagi sistem client-server dan aplikasi paralel.

```
suci@LAPTOP-2K58MOR4:~/praktikum_os$ gcc -o procfs_read procfs_read.c
./procfs_read
=== CPU Information ===
processor      : 0
vendor_id     : AuthenticAMD
cpu family    : 23
model         : 24
model name    : AMD Ryzen 3 3250U with Radeon Graphics
stepping      : 1
microcode     : 0xffffffff
cpu MHz       : 2599.920
cache size    : 512 KB
physical id    : 0

=== Memory Information ===
MemTotal:     2946292 kB
MemFree:      2533824 kB
MemAvailable: 2591120 kB
Buffers:      2180 kB
Cached:       112348 kB

=== Process Information ===
Name:   procfs_read
Umask:  0022
State:  R (running)
Tgid:   566
Ngid:   0
Pid:    566
PPid:   300
TracerPid: 0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
suci@LAPTOP-2K58MOR4:~/praktikum_os$ |
```

---

## Praktik 9.1 – Kernel Modules

Praktik ini memperkenalkan cara membuat *Linux Kernel Module (LKM)* sederhana. Mahasiswa menulis file `hello_module.c`, membuat `Makefile`, dan mengompilasinya menjadi file `.ko`. Modul tersebut dapat dimuat ke kernel dengan `insmod` dan dihapus dengan `rmmmod`. Dengan `dmesg`, hasil keluaran modul dapat dilihat di log kernel.

```
suci@LAPTOP-2K58MOR4:~/praktikum_os$ dmesg | less
systemd-analyze
systemd-analyze blame
systemd-analyze critical-chain
cat /boot/grub/grub.cfg | head -50
lsinitramfs /boot/initrd.img-$(uname -r) | head -20
Startup finished in 6.306s (userspace)
graphical.target reached after 6.239s in userspace.
2.994s landscape-client.service
1.565s snapd.seeded.service
1.550s dev-sdd.device
1.256s snapd.service
1.000s wsl-pro.service
652ms rsyslog.service
622ms systemd-udev-trigger.service
596ms systemd-resolved.service
434ms systemd-logind.service
410ms e2scrub_reap.service
396ms systemd-timesyncd.service
389ms user@1000.service
356ms dev-hugepages.mount
353ms systemd-journal-flush.service
337ms dev-mqueue.mount
332ms systemd-journald.service
332ms sys-kernel-debug.mount
328ms keyboard-setup.service
306ms sys-kernel-tracing.mount
278ms modprobe@configfs.service
272ms modprobe@dm_mod.service
268ms kmod-static-nodes.service
245ms systemd-udev.service
```

## Praktik 10.1 – Using strace

`strace` digunakan untuk melacak *system call* yang dijalankan program. Dengan perintah seperti `strace ls` atau `strace -o trace.txt ls`, mahasiswa dapat melihat aktivitas kernel saat menjalankan perintah. Sangat berguna untuk menganalisis error atau proses yang macet.

```
suci@LAPTOP-2K58MOR4:~/praktikum_os$ ps aux | grep ls
suci 1023 0.0 0.0 4092 1928 pts/0 S+
suci@LAPTOP-2K58MOR4:~/praktikum_os$ sudo strace -p 17
strace: attach: ptrace(PTRACE_SEIZE, 1234): No such pr
suci@LAPTOP-2K58MOR4:~/praktikum_os$ ^C
suci@LAPTOP-2K58MOR4:~/praktikum_os$ sleep 100
suci@LAPTOP-2K58MOR4:~/praktikum_os$

suci@LAPTOP-2K58MOR4:~$ cd ~/praktikum-os
-bash: cd: /home/suci/praktikum-os: No such file or directory
suci@LAPTOP-2K58MOR4:~$ cd ~/praktikum_os
suci@LAPTOP-2K58MOR4:~/praktikum_os$ ps aux | grep sleep
suci 1877 0.0 0.0 4092 1928 pts/2 S+ 17:17 0:00 grep --color=auto sleep
suci@LAPTOP-2K58MOR4:~/praktikum_os$
```



---

## Praktik 10.2 – Using ltrace

`ltrace` melacak pemanggilan fungsi dari pustaka (*library calls*). Dengan `ltrace` `ls`, dapat dilihat fungsi dari library C (`libc`) yang digunakan selama eksekusi program. Ini membantu memahami hubungan antara program pengguna dan library sistem.

```
suci@LAPTOP-2K58MOR4:~/praktikum_os$ gdb ./buggy
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./buggy...
(gdb) break main
Breakpoint 1 at 0x1238: file buggy.c, line 17.
(gdb) run
Starting program: /home/suci/praktikum_os/buggy
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at buggy.c:17
17      buggy_function(5);
(gdb) next
array[0] = 0
array[1] = 2
array[2] = 4
array[3] = 6
array[4] = 8
18      return 0;
(gdb) step
```

---

## Praktik 10.3 – Using GDB

Mahasiswa menggunakan *GNU Debugger (GDB)* untuk menemukan bug pada program `buggy.c`. Dengan perintah seperti `break`, `run`, `next`, dan `backtrace`, mahasiswa dapat menelusuri baris kode penyebab kesalahan. GDB sangat berguna untuk debugging tingkat lanjut.

---

## Praktik 10.4 – Kernel Debugging dengan dmesg

`dmesg` digunakan untuk membaca pesan kernel. Melalui log ini, pengguna dapat melihat pesan error, peringatan, dan status perangkat keras. Praktik ini membantu memahami interaksi langsung antara kernel dan perangkat komputer.

```
[ 12.945188] kvm_amd: kvm_amd: Hyper-V Direct TLB Flush enabled
[ 12.945198] kvm_amd: Virtual VMLOAD VMSAVE supported
[ 17.783436] systemd-journald[52]: /var/log/journal/889d8064ce14b0caa7530438b410612/user-1000.journal: Journal file uses a different sequence number ID, rotating.
[ 49.650466] hv_balloon: Max. dynamic memory size: 3040 MB
[ 613.574709] mini_init (174): drop_caches: 1
[ 986.613975] mini_init (174): drop_caches: 1
[ 1037.119744] TCP: eth0: Driver has suspect GRO implementation, TCP performance may be compromised.
[ 1118.516700] mini_init (174): drop_caches: 1
[ 1698.211721] mini_init (174): drop_caches: 1

^C
[ 0.137591] PCI: Fatal: No config space access function found
[ 5.839406] /dev/sdd: Can't open blockdev
[ 9.930219] WSL (190) ERROR: CheckConnection: getaddrinfo() failed: -5
[ 10.614322] misc dxg: dxgk: dxgkio_is_feature_enabled: Ioctl failed: -22
[ 10.637646] misc dxg: dxgk: dxgkio_query_adapter_info: Ioctl failed: -22
[ 10.662490] misc dxg: dxgk: dxgkio_query_adapter_info: Ioctl failed: -22
[ 10.664593] misc dxg: dxgk: dxgkio_query_adapter_info: Ioctl failed: -22
[ 10.670892] misc dxg: dxgk: dxgkio_query_adapter_info: Ioctl failed: -22
[ 10.678478] misc dxg: dxgk: dxgkio_query_adapter_info: Ioctl failed: -2
[ 0.101567] Speculative Return Stack Overflow: IBPB-extending microcode not applied!
[ 0.101568] Speculative Return Stack Overflow: WARNING: See https://kernel.org/doc/html/latest/admin-guide/hw-vuln/srso.html for mitigation options.

ns.
[ 0.171805] PCI: System does not support PCI
[ 0.682066] device-mapper: core: CONFIG_IMA_DISABLE_HTABLE is disabled. Duplicate IMA measurements will not be recorded in the IMA log.
[ 9.978792] pulseaudio[227]: memfd_create() called without MFD_EXEC or MFD_NOEXEC_SEAL set
[ 9.991088] Failed to connect to bus: No such file or directory
[ 10.318126] Failed to connect to bus: No such file or directory
[ 10.628807] Failed to connect to bus: No such file or directory
[ 11.714229] systemd-journald[52]: File /var/log/journal/889d8064ce14b0caa7530438b410612/system.journal corrupted or uncleanly shut down, renaming and replacing.
[ 1037.119744] TCP: eth0: Driver has suspect GRO implementation, TCP performance may be compromised.
suci@LAPTOP-2K58MOR4:~/praktikum_os$
```

## Praktik 10.5 – Memory Leak Detection dengan Valgrind

Praktik ini menggunakan `valgrind` untuk mendeteksi kebocoran memori dalam program. Jika `malloc()` digunakan tanpa `free()`, maka `valgrind` akan menampilkan laporan memori yang tidak dibebaskan. Ini penting untuk menjaga efisiensi program dan mencegah crash.

## Praktik 10.6 – Performance Analysis

Menggunakan `perf` untuk menganalisis performa sistem dan aplikasi. Dengan `perf record` dan `perf report`, mahasiswa dapat memantau aktivitas CPU, *system call*, dan beban proses. Alat ini berguna untuk melakukan *profiling* dan optimasi performa aplikasi.